

## A2 · Answer Key and Demo Script

CS425 · Stop Typing Your Password · instructor led

INSTRUCTOR COPY

This one is a follow along, not a group activity: you do each step on the projector, say "hands up when", and wait. Every transcript below is real output captured from a laptop and from `onyx.boisestate.edu` while this was written. Numbers will differ; the shapes will not.

### Say this at the start

"Last week you typed your password a dozen times. By the end of today you will never type it again, and you will have a command called `probe` that you will use for the next two activities. Follow along; I will wait at every step." Then hand out the sheet and start step 1 on the projector.

### BUDGET

| STEPS                                              | MINUTES | WHAT TO WAIT FOR                                                                                             |
|----------------------------------------------------|---------|--------------------------------------------------------------------------------------------------------------|
| 0, password login                                  | 10      | Every hand up for a prompt on Onyx. One student may take the whole period; do not let the room wait on them. |
| 1 to 2, key and copy                               | 15      | Every hand up for <code>ssh user@onyx uname -n</code> with no password. This is where Windows eats time.     |
| 3 to 4, config and timing                          | 10      | Four rows of timings on every sheet.                                                                         |
| 5 to 7, <code>bashrc</code> and <code>probe</code> | 15      | <code>ssh onyx probe example.com 80</code> working from a laptop.                                            |
| 8, alias                                           | 5       | Q3 answered.                                                                                                 |
| 9, laptop                                          | rest    | Optional; A4 goes faster for anyone who finishes it.                                                         |

### STEP 0 · CAN THEY GET THERE AT ALL

| MESSAGE                                | WHERE IT STOPPED                  | FIX                                                                                                |
|----------------------------------------|-----------------------------------|----------------------------------------------------------------------------------------------------|
| command not found / not recognized     | The laptop; no client on the path | Git Bash. Do not install anything in class.                                                        |
| Could not resolve hostname             | Name resolution                   | Spelling, then <code>curl -I https://www.boisestate.edu/</code> to prove the Wi-Fi works.          |
| Connection timed out                   | The network; nothing answered     | Phone hotspot. If that works, the first network filters 22.                                        |
| Connection refused                     | A host, but not Onyx              | Wrong hostname, nearly always.                                                                     |
| Permission denied (publickey,password) | Onyx; the account                 | Username, not email. Slow retype. Three strikes: probably no Onyx account, note the name, move on. |
| REMOTE HOST IDENTIFICATION HAS CHANGED | Stale fingerprint on the laptop   | <code>ssh-keygen -R onyx.boisestate.edu</code> , then yes.                                         |

**The universal answer:** `ssh -v user@onyx.boisestate.edu` and read the last lines before it stopped. Connecting to with nothing after it is the network; Authentications that can continue then a failure is the account. Put that on the board and leave it there; it is the one thing from today that transfers to every other host.

## STEPS 1 TO 4 · KEYS AND MULTIPLEXING

### Cold versus warm, measured from a laptop

```
$ ssh -O exit onyx ; time ssh onyx true
cold (full handshake): 1105 ms
```

```
$ time ssh onyx true
warm (reused master): 248 ms
```

```
$ ping -c 3 onyx.boisestate.edu
rtt min/avg/max = 31.4/33.7/35.3 ms
```

SAY: about 1.1 seconds against a 31 ms round trip. Ask what the extra time bought: a TCP handshake, a version exchange, a key exchange, and authentication. Accept any estimate in the range of a handful to a few dozen round trips. **DO NOT LET THEM CONCLUDE IT IS ALL NETWORK TIME:** key exchange is arithmetic, and on a fast link the crypto dominates. The warm case is roughly 4x faster and still not zero, because starting a process on the far end is not free.

**Q1.** Any estimate from about 5 to about 30 round trips passes, with something named: handshake, version exchange, key exchange, authentication. The largest speedup is normally the laptop with the slowest link or the slowest CPU; either explanation is fine if it matches their numbers.

### Where step 2 goes wrong

- **No ssh-copy-id on Windows.** Manual procedure is on the page; have it on the board before you start step 2, not after the first hand goes up.
- **Still asks for a password.** Permissions, nearly always. On Onyx: `chmod 700 ~/.ssh; chmod 600 ~/.ssh/authorized_keys`. On the laptop: `chmod 600 ~/.ssh/config ~/.ssh/id_ed25519`. Run `ssh -v onyx` and read the line after "Offering public key".
- **Windows and ControlMaster.** Multiplexing is unreliable on Windows builds of OpenSSH. A Git Bash student may see a warning or two cold timings. Write both down, note the OS, move on.

## STEPS 5 TO 8 · THE FUNCTION

### Loaded and proven

```
$ cat ~/.bash_profile
if [ -f ~/.bashrc ]; then
    . ~/.bashrc
fi
                                     ← the login shell sources .bashrc
```

```
$ head -20 ~/.bashrc
(no [ -z "$PS1" ] line on Onyx)
```

```
$ type probe
probe is a function
```

```
$ probe example.com 80
connect=0.033370 total=0.057519 code=200
```

```
$ ssh onyx probe example.com 80      # non-interactive, from the laptop
connect=0.033370 total=0.057519 code=200
```

SAY: the last one is the one that matters. That shell had no prompt and no terminal, and the function was still there, because bash reads `~/.bashrc` when a remote shell daemon starts it and Onyx has no guard line stopping it. **VERIFIED ON ONYX**, so if a student's fails, the cause is in their file, not in the machine. Usual causes: the edit did not save, or it went into a different file.

**Q2.** A guard line returns from `~/.bashrc` before the function is defined whenever the shell is not interactive, and `ssh host` command is not interactive, so the function never exists in that shell. Any sentence with "not interactive" and "returns before" or "never reaches" passes.

**Q3.** An alias is textual substitution at the front of a command, so the words you type can only land at the end. There is no way to put the second word into the middle of a URL. A function has positional parameters, `$1` and `$2`, and that is the whole reason `probe` is a function.

## Rescue

- A **broken** `~/ .bashrc` can cost a student their shell. `ssh onyx -t 'bash --norc'` gets them back in to fix it.
- **vi.** On the board before step 3 and left there: `i` to type, `ESC` to stop, `:wq` save and quit, `:q!` bail out, `G` end of file, `o` new line below. Typed before `i` and things went strange: `ESC`, `:q!`, start over.
- **Missed the class.** The page is written to be done alone; point them at it and at A3's note that **probe** can be pasted at the prompt for one session.

**PASS BAR:** a row per member with two timings, and something written under each of the three questions. Award 20 or nothing.